People's Democratic Republic of Algeria
Ministry of Higher Education and Scientific Research

University of Belhadj Bouchaib – Ain Témouchent

Faculty of Science and Technology
Department of Mathematics and Computer Science

Project Network Programming

Mini-Chat client/server (TCP-UDP)

Done and Presented by:

HASSI Imad-Eddin — grp:1

TAYEB BENYAHIA Souad — grp:2

Professor:
Dr. BOUAAFIA

Academic Year: 2025 – 2026

Contents

1 Introduction **2**
1.1 Project Goal & Scope 2
1.2 Technologies Used 2
1.3 Report Structure Overview 3

2 Technical Choices & Architecture **3**
2.0.1 The Role of TCP (Transmission Control Protocol) 3

3 System Design & Modeling **4**
3.1 Use Case Diagrams 4
3.1.1 Client-side Use Cases 4
3.1.2 Server-side Use Cases 5
3.2 Sequence Diagrams 5
3.2.1 Client Connection & Disconnection 5
3.2.2 Message Sending & Receiving 7
3.2.3 username Change Process 7
3.2.4 Fetching the Connected Users List 8

4 Implementation & Screenshots **9**
4.1 Server Architecture 9
4.2 Client Logic 9
4.3 Visual Demonstration (Screenshots) 10

5 Challenges & Solutions **13**
5.1 Challenge: Firewall Blocking Server-Side Incoming Connections 13

6 Conclusion **13**

1 Introduction

1.1 Project Goal & Scope

The objective of this project is the design and development of a high-performance, real-time multi-client chat application using the Java networking stack. Operating on a robust **TCP-based client-server model**, the system is engineered to support concurrent communication between multiple users through a centralized server hub.

The scope of the project extends beyond basic text exchange to include a comprehensive suite of modern messaging features. Core functionalities implemented include:

- **Dynamic Session Management:** Handling asynchronous client connections and disconnections via multi-threading.
- **Identity & Customization:** A dedicated registration system with persistent user storage and the ability to select or modify visual avatars and pseudonyms.
- **Advanced Protocol Interaction:** A command-driven interface allowing users to query server state (e.g., retrieving active user lists).
- **Data Exchange:** A simulated file-sharing protocol that permits the "upload" and "download" of assets within a shared network directory.

1.2 Technologies Used

The implementation of the Mini-Chat system relies on several core technologies to ensure reliable communication and an intuitive user experience:

- **TCP (Transmission Control Protocol):** The network communication architecture exclusively utilizes TCP to provide a connection-oriented environment. This protocol is leveraged for stateful interactions requiring guaranteed delivery, ensuring that neither chat messages nor binary file data are lost or corrupted during transit.
- **Java Standard Edition:** The application is developed entirely in **Java**, utilizing its robust `java.net` library for socket programming and `java.io` for handling data streams between the client and the server.
- **Swing Library:** The client-side graphical user interface (GUI) is built using the **Swing** framework. This allows for a multi-windowed experience, including dedicated forms for registration, server connection, and the main chat salon.
- **Multithreading:** To manage concurrent sessions, the system uses Java's `Thread` class. This allows the server to process multiple clients simultaneously and enables the client to listen for incoming messages while maintaining a responsive UI.

1.3 Report Structure Overview

The logical progression of this report follows the standard software engineering lifecycle, beginning with the conceptual modeling of the system through Use Case and Sequence diagrams. It then transitions into the technical architecture, detailing the multi-threaded server-side logic and the dual-threaded client-side implementation required for a responsive GUI. Finally, the implementation is validated through a visual demonstration of core features—such as real-time broadcasting, command execution, and file sharing—concluding with an analysis of the technical challenges encountered and the networking solutions engineered to overcome them.

2 Technical Choices & Architecture

2.0.1 The Role of TCP (Transmission Control Protocol)

TCP was chosen for all critical and stateful operations where data integrity and order are paramount. Its connection-oriented and reliable nature makes it the only suitable choice for:

- Initial client authentication and connection setup.
- The transmission of all chat messages to ensure no messages are lost or received out of order.
- The execution of specific commands, such as changing a nickname (‘/nick‘) or requesting the user list (‘/list‘).

For these core features, the guarantees offered by TCP’s three-way handshake and acknowledgment mechanisms are non-negotiable.

3 System Design & Modeling

The application's conceptual model is defined using standard UML diagrams. Use Case diagrams outline the system's functionalities from the perspective of its actors. Sequence diagrams then illustrate the key interaction scenarios step-by-step.

3.1 Use Case Diagrams

The system's functionalities are defined for two primary actors: the user (client-side) and the server system.

3.1.1 Client-side Use Cases

The `User` is the primary actor. Their main interactions involve connecting to the server, sending and receiving messages, changing their pseudonym, requesting the list of connected users, and disconnecting .

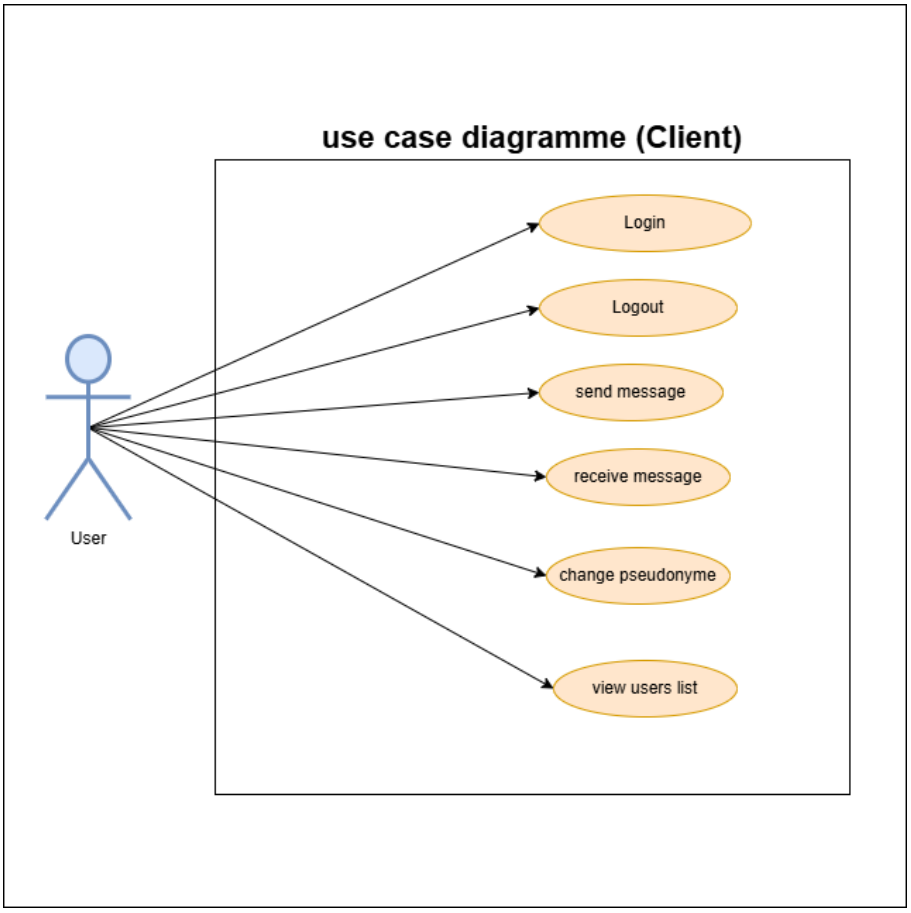

Figure 1: Client use case diagramme

3.1.2 Server-side Use Cases

From the server's perspective, the `Server System`'s main roles are to manage user connections, broadcast messages, update the user list, and handle pseudonym changes. These processes are initiated by a `Client` actor .

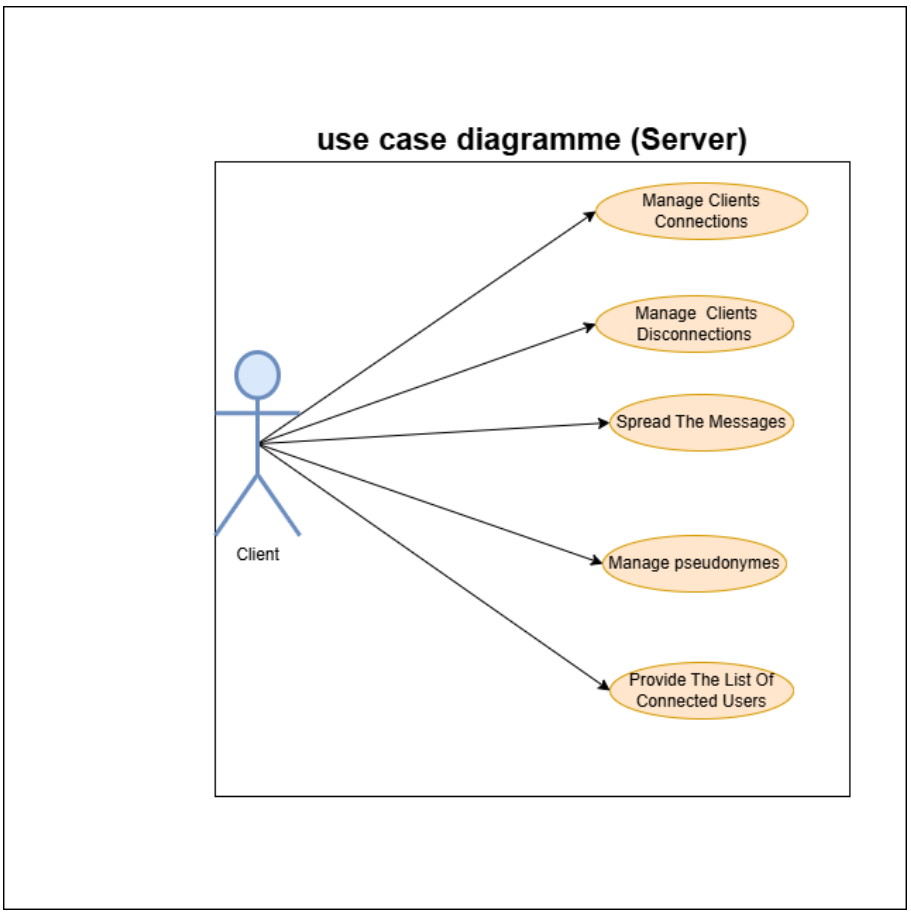

Figure 2: Server use case diagram

3.2 Sequence Diagrams

The interactions between the client, the server, and the server's dedicated client threads are detailed for the most important scenarios.

3.2.1 Client Connection & Disconnection

The connection process is initiated by the client. The server accepts the request and spawns a dedicated thread to handle all future communications with that client .

Disconnection is also client-initiated. A request is sent to the server thread, which then closes the socket and terminates, ending the session .

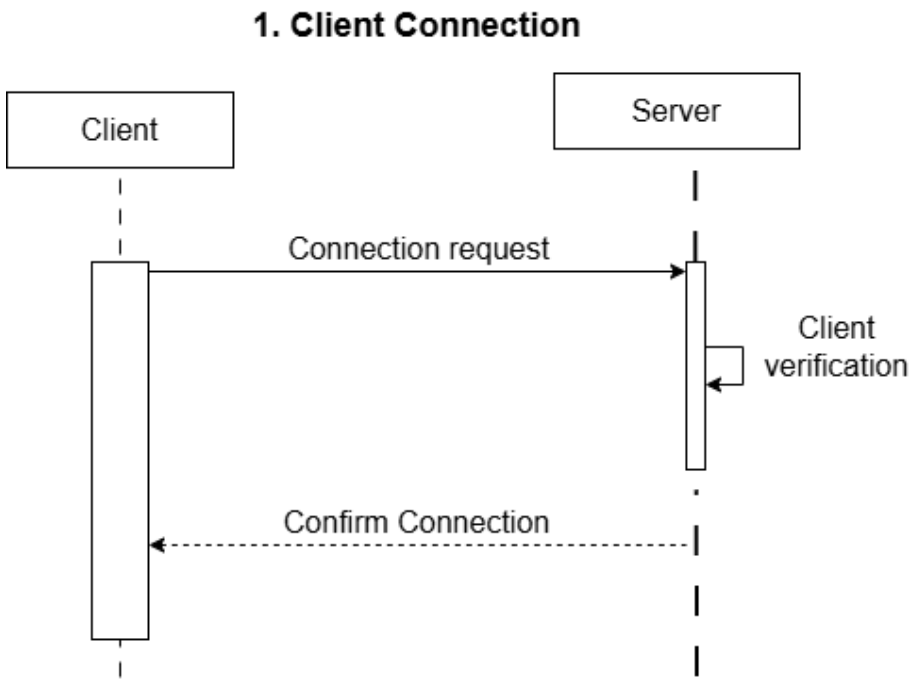

Figure 3: Client connection

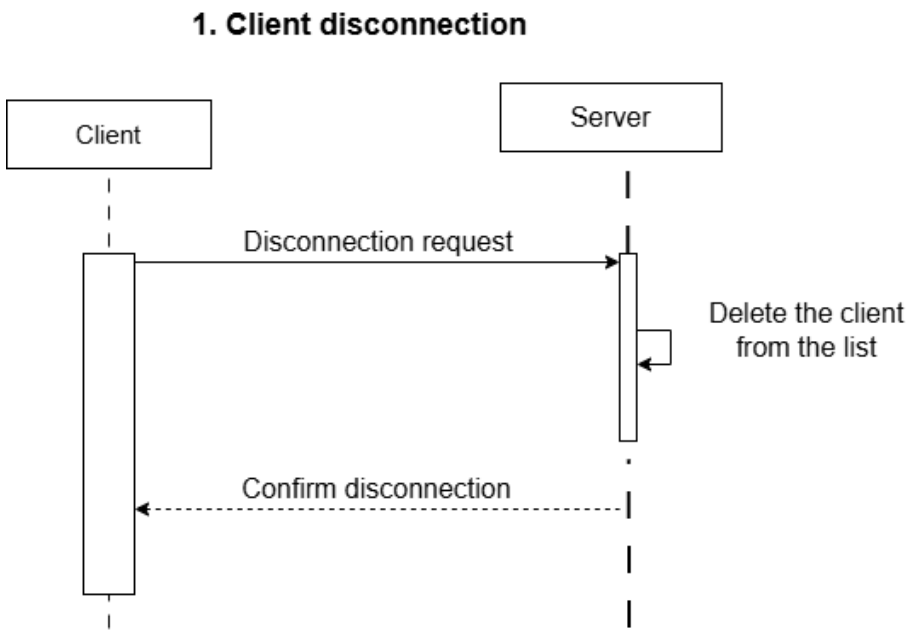

Figure 4: Client Disconnection

3.2.2 Message Sending & Receiving

A message sent from a client application is first received by its dedicated server thread. That thread then broadcasts the message to all other client threads, which in turn forward it to their respective client applications for display .

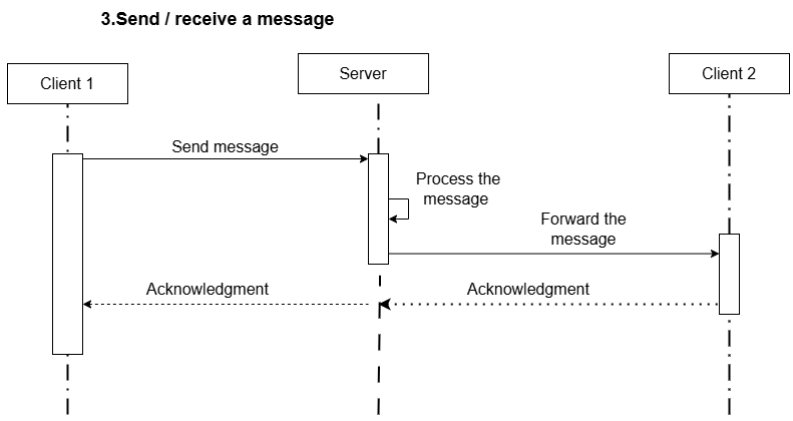

Figure 5: Send / Receive

3.2.3 username Change Process

A client requests a pseudonym change by sending a specific command to the server. The corresponding server thread validates the request and updates the user's information. A confirmation is returned to the original client, and an update is forward to all others .

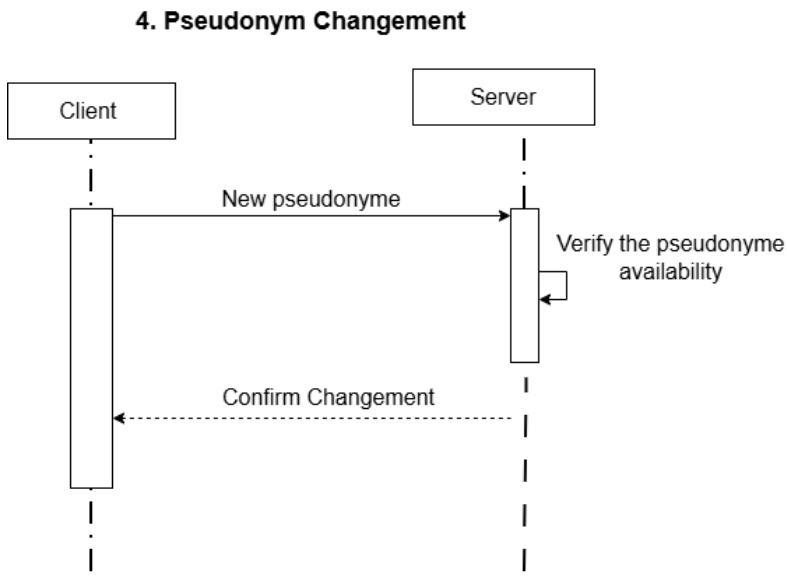

Figure 6: Pseudonym Change

3.2.4 Fetching the Connected Users List

To retrieve the list of all connected users, the client sends a specific request command. The server thread compiles the current list of active users and sends it back directly to the requesting client (Figure **??**).

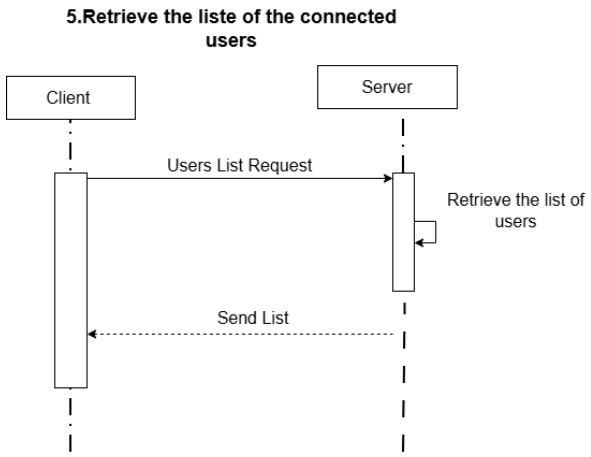

Figure 7: Users list

4 Implementation & Screenshots

4.1 Server Architecture

The server is built using a **Multi-threaded TCP architecture** to handle concurrent client sessions. It initializes a `ServerSocket` on port 2000 and enters an infinite loop to await incoming connection requests.

For every successful connection, the server instantiates a `ChatService` object, which implements the `Runnable` interface. This object is then passed to a new `Thread` and started, allowing the server to offload communication logic—such as authentication, broadcasting messages, and handling slash commands like `/logout`—to a dedicated worker thread. This design ensures that the main server thread remains free to accept new users without being blocked by the activity of existing ones.

4.2 Client Logic

The client-side application employs a **dual-threaded approach** to maintain a responsive Graphical User Interface (GUI).

- **Main UI Thread:** Handles user interactions, such as clicking buttons to send messages, selecting avatars, or triggering file uploads and downloads.
- **Receiver Thread:** Upon initialization, the `Chat` class spawns a background thread dedicated to listening for data from the server. It uses a `BufferedReader` to read incoming strings in a continuous loop. When a message or a system notification is received, the thread updates the `jTextArea1` component in real-time.

This separation prevents the GUI from "hanging" while waiting for network responses, providing a smooth user experience.

4.3 Visual Demonstration (Screenshots)

The following figures provide visual proof of the application's functionality, from the initial server handshake to active communication.

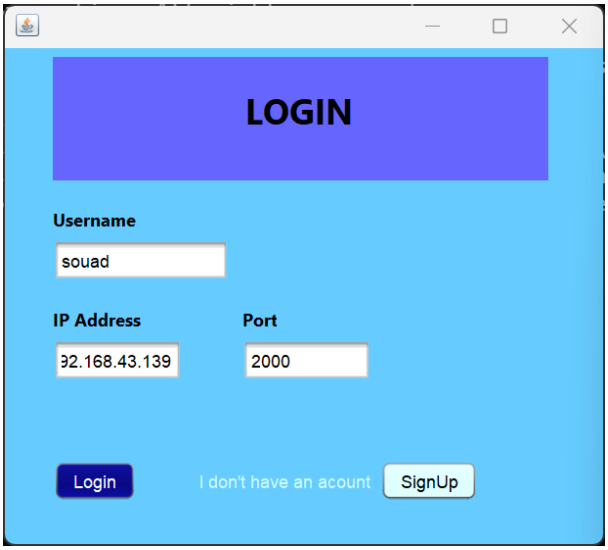

Figure 8: Login with IP ADD

Figure 9: Login with localhost

"This screenshot demonstrates the application running in a local loopback environment. By using the address 192.168.43.139 (or localhost), the client connects to a server instance running on the same physical machine. This is the primary method used for debugging and initial testing of the socket handshake protocol without requiring an external network."

The application supports dynamic user state changes and administrative commands through the chat interface.

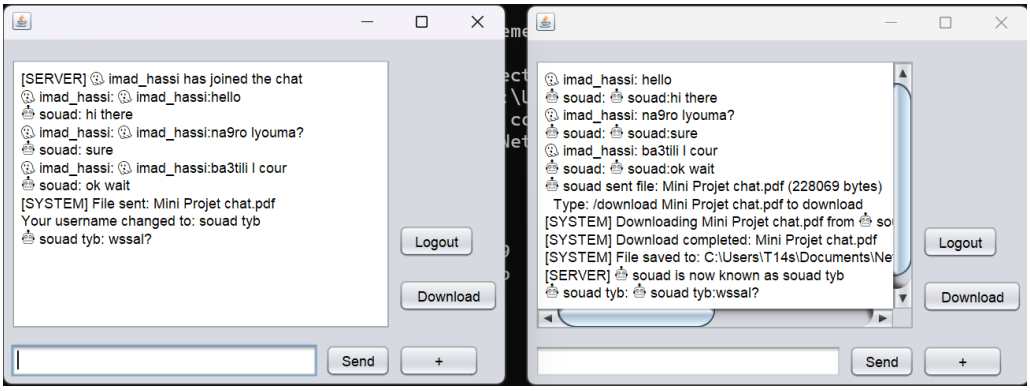

Figure 10: chat and exchange file and username change

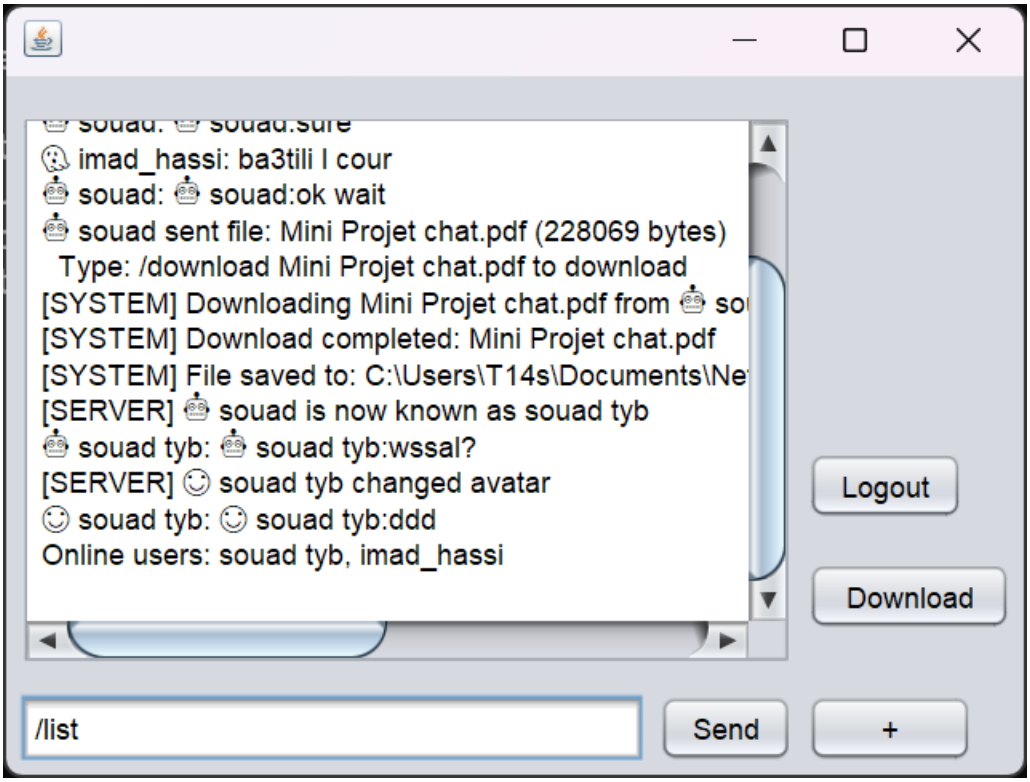

Figure 11: command to get users list

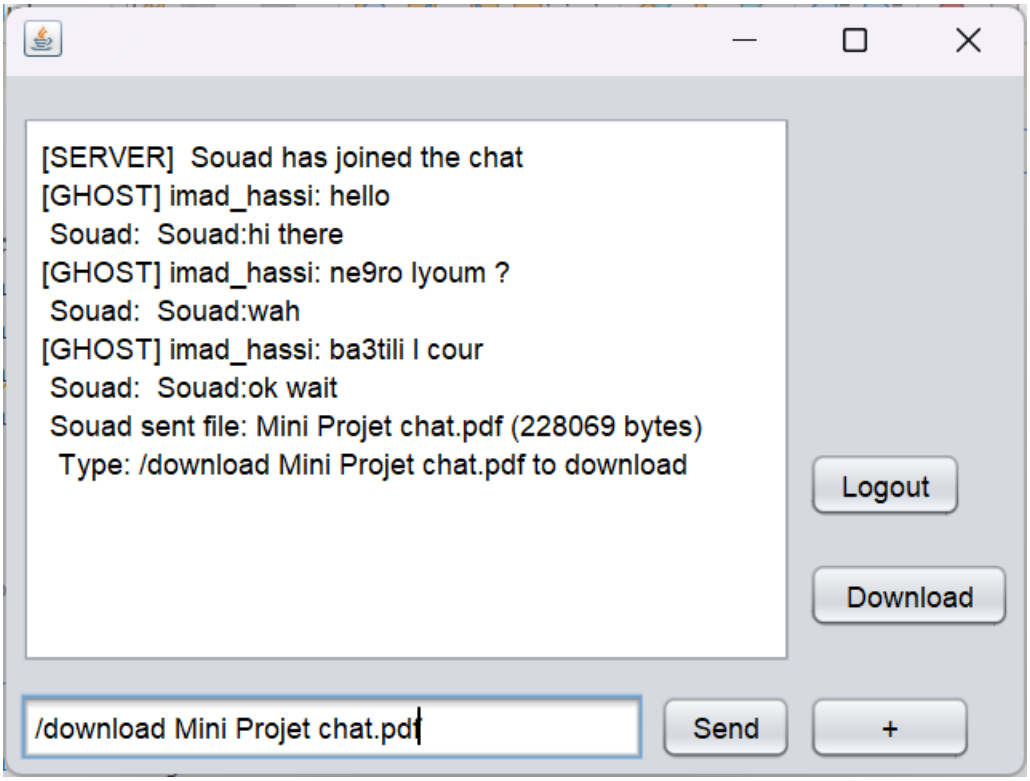

Figure 12: download Files

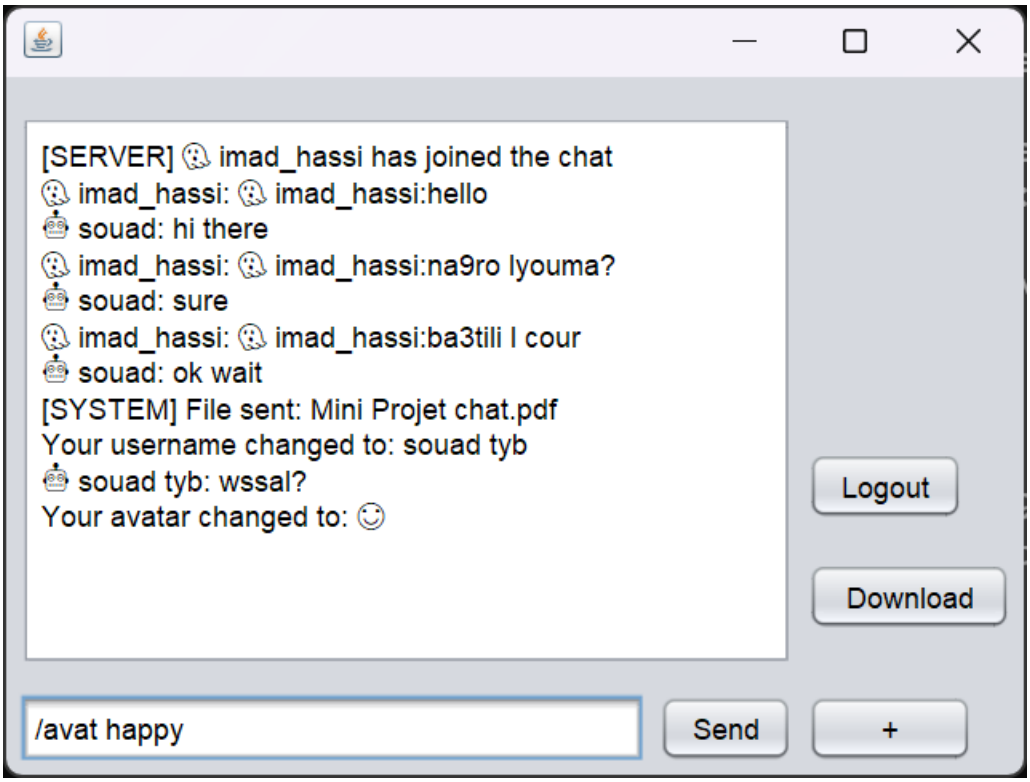

Figure 13: change avatar with command /avat

5 Challenges & Solutions

During development,There was several technical challenges.

5.1 Challenge: Firewall Blocking Server-Side Incoming Connections

Problem: During testing, it was discovered that clients on different machines could not connect to the server, even when using the correct IP address and port. The connection attempts would consistently time out. The issue was identified as the host machine's software firewall which, by default, blocks all unsolicited incoming network traffic on non-standard ports. This security feature prevents the client's initial TCP connection request from ever reaching the server application.

Solution: The solution is not programmatic but rather operational, requiring a manual configuration step on the machine hosting the server. To resolve the issue, a new **inbound rule** must be created in the host's firewall settings. This rule must be configured to explicitly **allow incoming TCP traffic** on the specific port number the server application is listening on (e.g., port 8080). Once this rule is enabled, the firewall permits the connection requests from clients, and the application functions as expected across the network.

6 Conclusion

The development of this Mini-Chat application successfully demonstrates the practical application of **Socket programming** and **Client-Server architecture** within the Java ecosystem. By utilizing the **TCP protocol**, the project ensures a reliable and ordered exchange of data, which is essential for both real-time text communication and the integrity of shared files.

From a technical standpoint, the implementation of a multi-threaded server proved to be the cornerstone of the system, allowing for concurrent user sessions and synchronized message broadcasting without performance bottlenecks. Furthermore, the integration of advanced functionalities—such as the command-driven protocol for retrieving active user lists (`/list`), dynamic identity management through avatars, and a functional file exchange system—elevates the application from a simple messaging tool to a comprehensive communication platform.

Ultimately, this project provided valuable insight into the challenges of network synchronization, state management, and GUI responsiveness. The successful deployment of the application across both local and network interfaces (LAN) confirms that the initial design objectives were met, providing a solid foundation for more complex distributed systems in the future.